Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure and Results

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Output

```
First five samples:
   Variance  Skewness  Curtosis  Entropy  Class
0   3.62160    8.6661   -2.8073 -0.44699      0
1   4.54590    8.1674   -2.4586 -1.46210      0
2   3.86600   -2.6383    1.9242  0.10645      0
3   3.45660    9.5228   -4.0112 -3.59440      0
4   0.32924   -4.4552    4.5718 -0.98880      0
```

```
Dataset dimensions (rows, columns): (1372, 5)

Missing values per column:
Variance    0
Skewness    0
Curtosis    0
Entropy     0
Class       0
dtype: int64

Descriptive statistics:
          Variance     Skewness     Curtosis      Entropy        Class
count  1372.000000  1372.000000  1372.000000  1372.000000  1372.000000
mean      0.433735     1.922353     1.397627    -1.191657     0.444606
std       2.842763     5.869047     4.310030     2.101013     0.497103
min      -7.042100   -13.773100    -5.286100    -8.548200     0.000000
25%      -1.773000    -1.708200    -1.574975    -2.413450     0.000000
50%       0.496180     2.319650     0.616630    -0.586650     0.000000
75%       2.821475     6.814625     3.179250     0.394810     1.000000
max       6.824800    12.951600    17.927400     2.449500     1.000000

Class distribution:
Class
0    762
1    610
Name: count, dtype: int64
```

Task 2: Exploratory Data Analysis

Generate Histograms, Correlation Heatmap, Scatter Plot, and Boxplots.

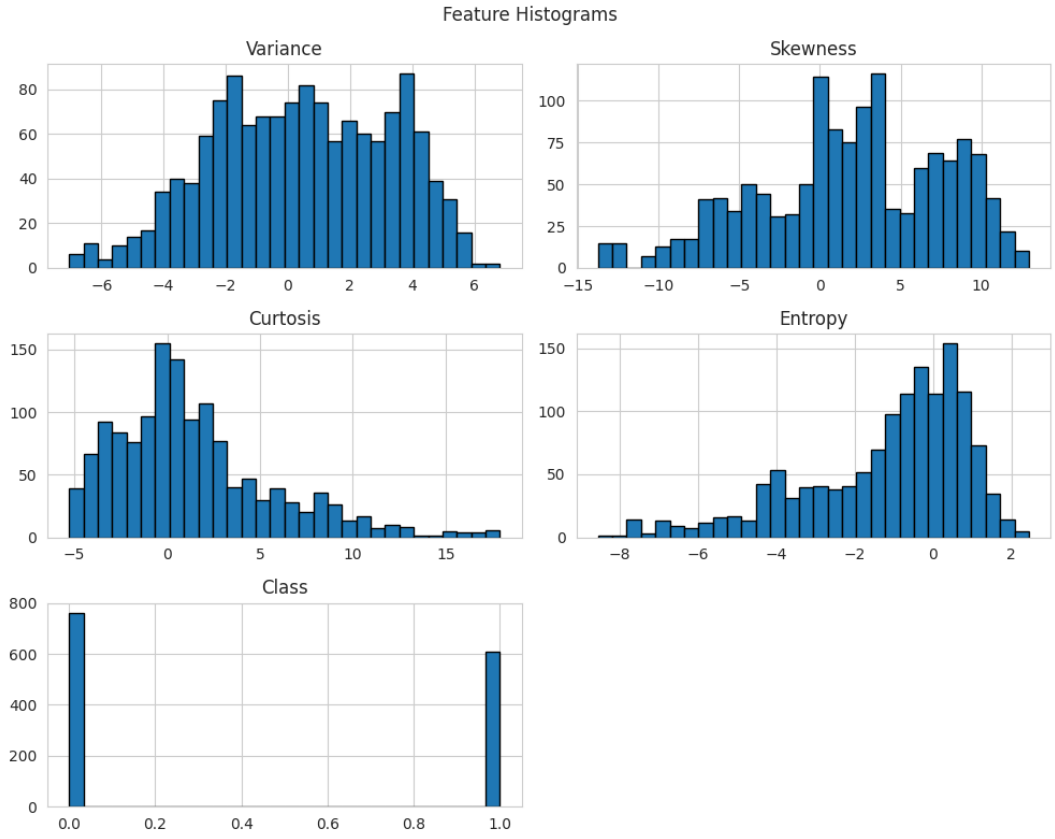

Figure 1: Feature Histograms

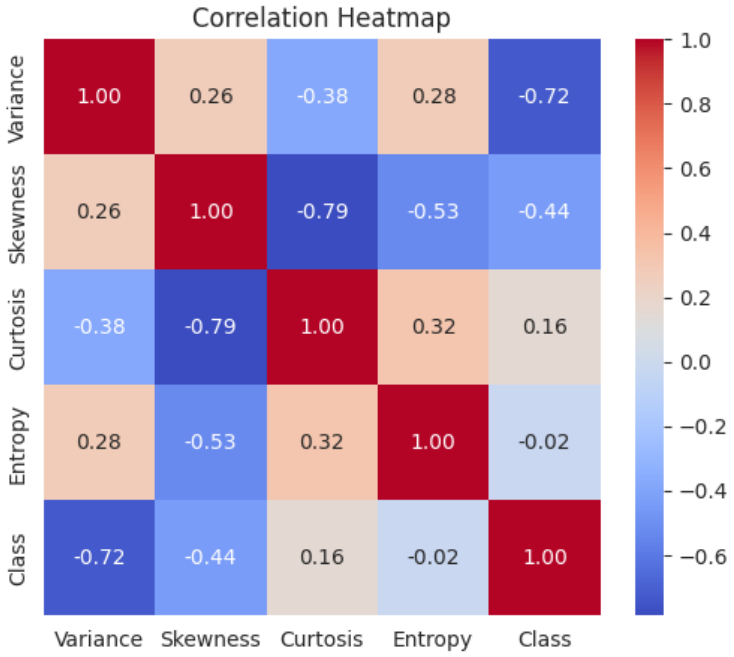

Figure 2: Correlation Heatmap

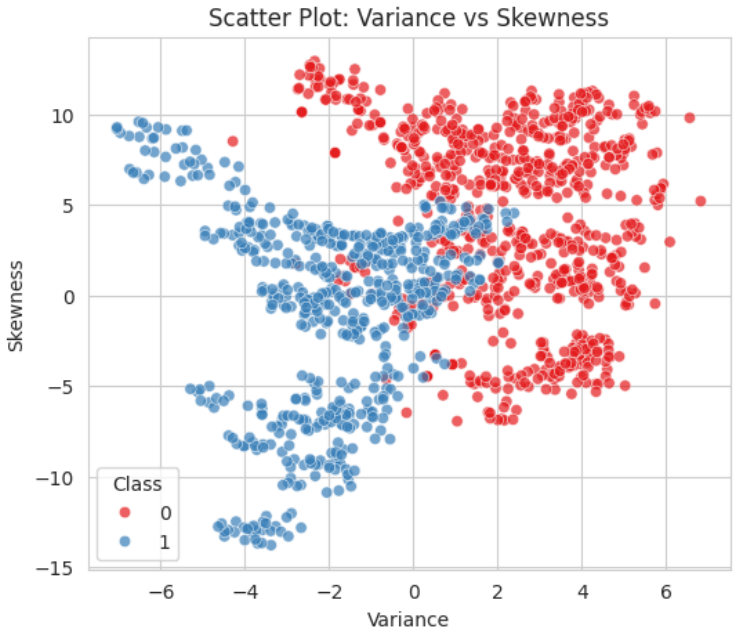

Figure 3: Scatter Plot: Variance vs Skewness

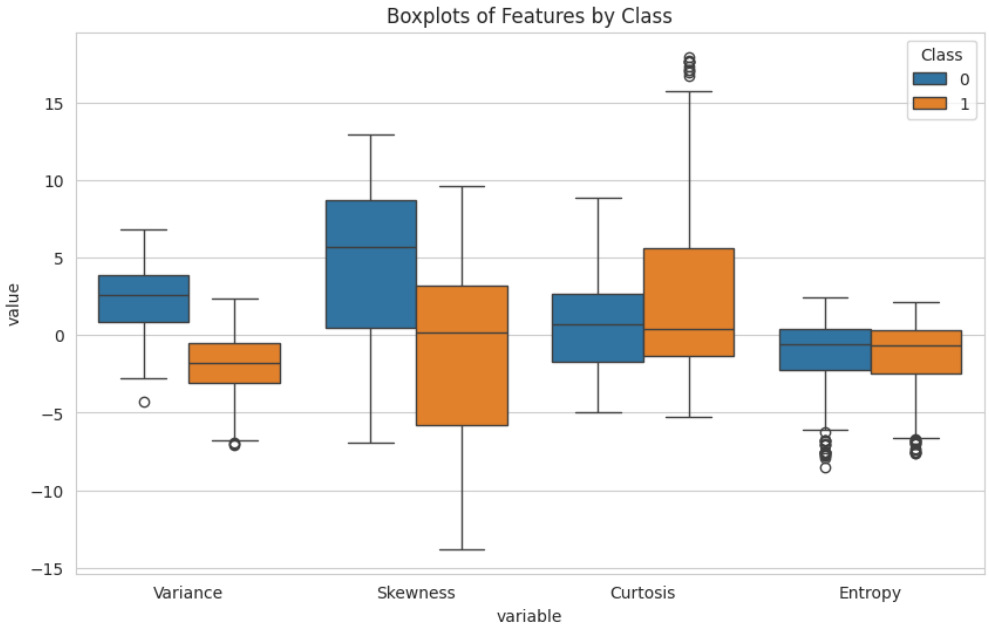

Figure 4: Boxplots of Features by Class

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Output

```
Training set size: (1097, 4)
Testing set size: (275, 4)
```

Task 4 & 5: Perceptron Implementation and Model Training

Implement from scratch: Weight Initialization, Bias Initialization, Step Activation Function, Forward Propagation, and the Perceptron Learning Rule. Display, per epoch: Epoch Number, Misclassified Samples, Updated Weights, Updated Bias.

```
Epoch   1 | Misclassified:   59 | Weights: [-0.068  -0.0938 -0.0678 -0.011 ] | Bias: -0.03
Epoch   2 | Misclassified:   25 | Weights: [-0.0775 -0.1042 -0.0811  0.0035] | Bias: -0.04
Epoch   3 | Misclassified:   20 | Weights: [-0.0802 -0.1127 -0.0917 -0.0036] | Bias: -0.04
Epoch   4 | Misclassified:   19 | Weights: [-0.0883 -0.1196 -0.1001  0.0089] | Bias: -0.05
Epoch   5 | Misclassified:   20 | Weights: [-0.0991 -0.1295 -0.1037  0.0102] | Bias: -0.05
...
Epoch  46 | Misclassified:   15 | Weights: [-0.1942 -0.2323 -0.2175 -0.0208] | Bias: -0.09
Epoch  47 | Misclassified:   16 | Weights: [-0.1987 -0.2279 -0.218  -0.0179] | Bias: -0.09
Epoch  48 | Misclassified:   17 | Weights: [-0.1951 -0.2336 -0.2113 -0.0119] | Bias: -0.10
Epoch  49 | Misclassified:   14 | Weights: [-0.2092 -0.2231 -0.2069 -0.0255] | Bias: -0.10
Epoch  50 | Misclassified:   18 | Weights: [-0.1941 -0.2358 -0.2109 -0.0195] | Bias: -0.10
```

Task 6: Model Evaluation

Compute Accuracy, Precision, Recall, F1-score, and the Confusion Matrix.

Output

```
Accuracy  : 0.9855
Precision : 0.9683
Recall    : 1.0000
F1-score  : 0.9839

Confusion Matrix:
[[149   4]
 [  0 122]]
```

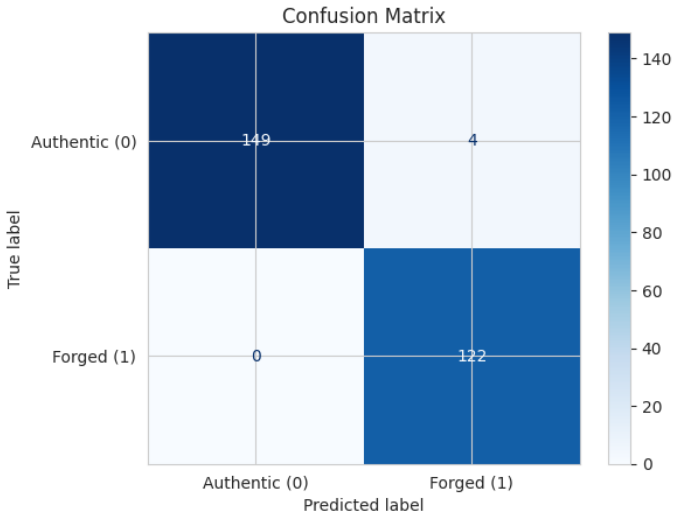

Figure 5: Confusion Matrix

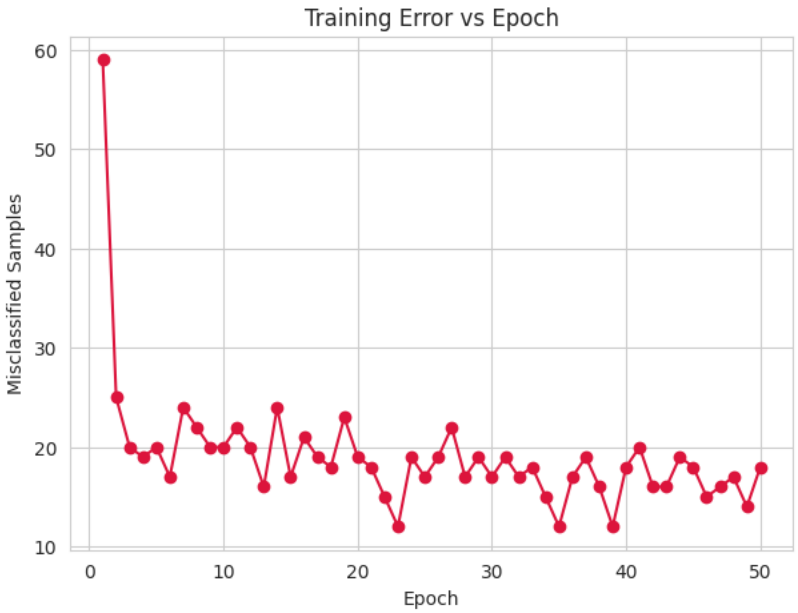

Figure 6: Training Error vs Epoch

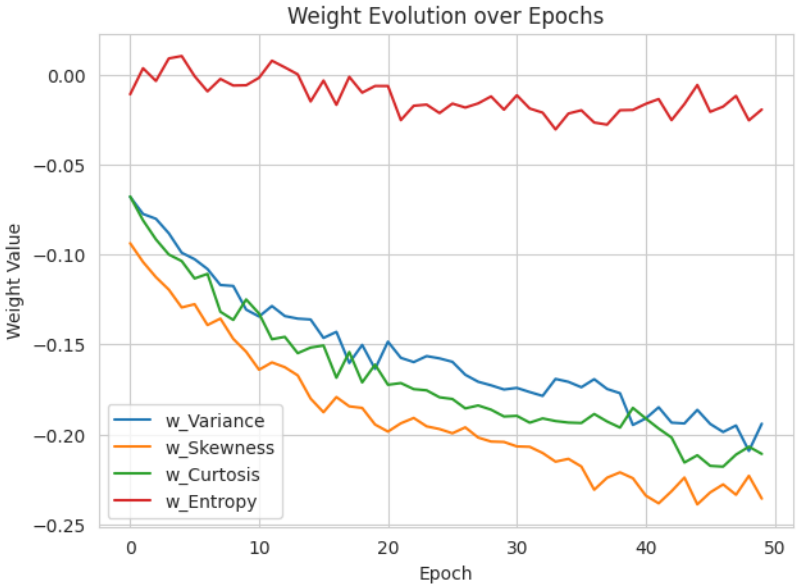

Figure 7: Weight Evolution over Epochs

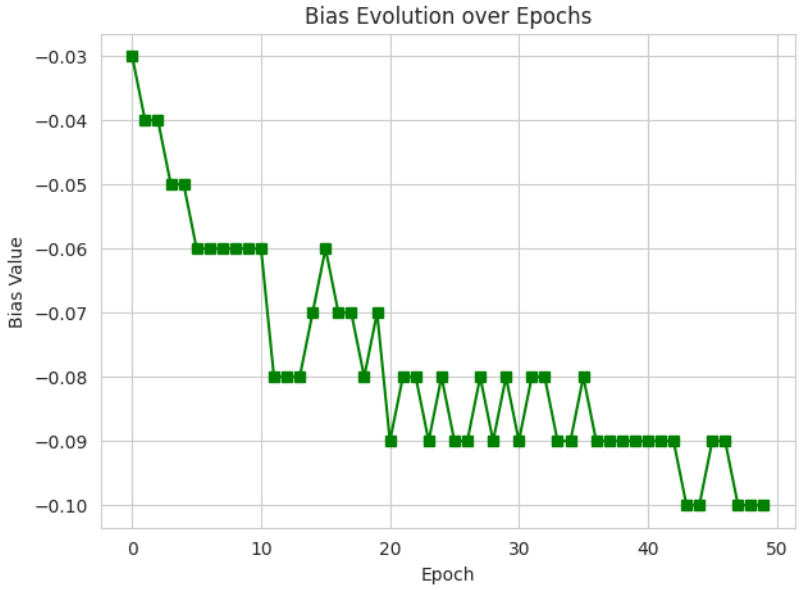

Figure 8: Bias Evolution over Epochs

Task 7: Effect of Different Learning Rates

Output

```
Learning Rate = 0.001  | Epochs run =  50 | Final Misclassified =   18 | Test Accuracy =
    0.9855
Learning Rate = 0.01   | Epochs run =  50 | Final Misclassified =   18 | Test Accuracy =
    0.9855
Learning Rate = 0.1    | Epochs run =  50 | Final Misclassified =   18 | Test Accuracy =
    0.9855
```

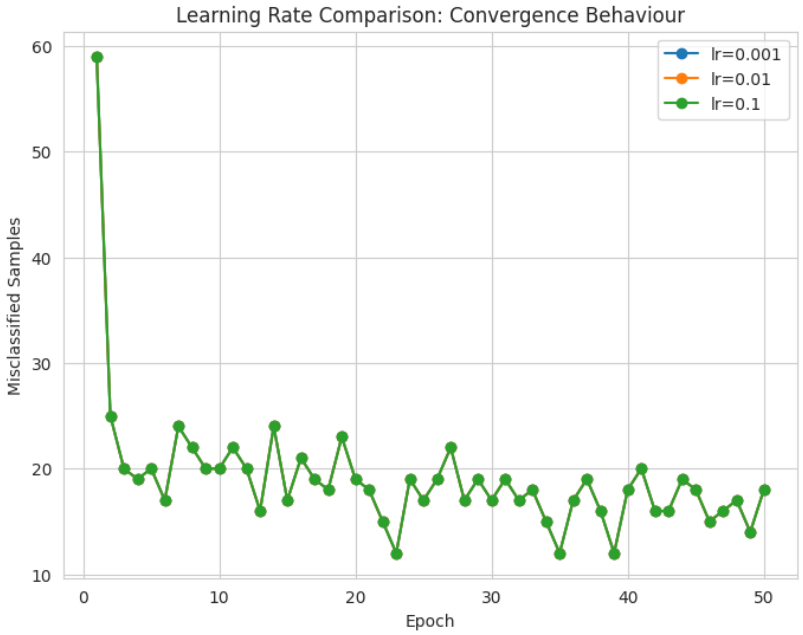

Figure 9: Learning Rate Comparison: Convergence Behaviour

Optional: Decision Boundary (Variance vs Skewness)

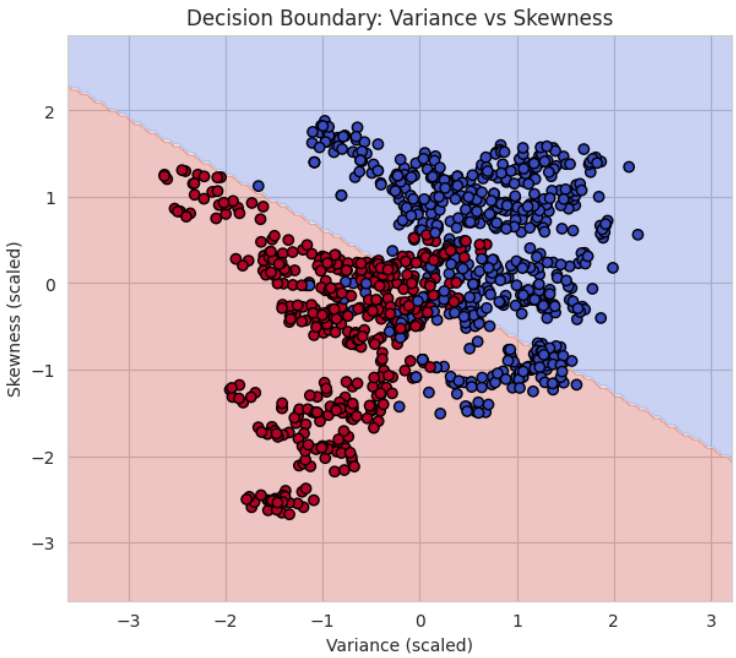

Figure 10: Decision Boundary: Variance vs Skewness

7. Mandatory Plots Summary

Plot	**Purpose**	**Expected Inference**
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Checks if classes are linearly separable.
Training Error vs Epoch	It learns behaviour	Verify convergence of the perceptron.
Weight Evolution	Parametric Analysis	Observe how stable the learned weights are.
Bias Evolution	Parametric Analysis	Does the role of bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

All plots above are generated and interpreted in-line under their respective tasks in Section 6.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	1097 / 275 (80% / 20%)
Learning Rate	0.01
Epochs	50
End Weights	$[-0.1941,\ -0.2358,\ -0.2109,\ -0.0195]$
End Bias	-0.10
Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839

Epoch-wise Learning (Learning Rate = 0.01)

Epoch	Errors	Weight 1 (Variance)	Weight 2 (Skewness)	Bias
1	59	−0.0680	−0.0938	−0.03
5	20	−0.0991	−0.1295	−0.05
10	20	−0.1307	−0.1541	−0.06
20	19	−0.1636	−0.1946	−0.07
30	17	−0.1751	−0.2043	−0.08
40	18	−0.1948	−0.2244	−0.09
45	18	−0.1864	−0.2389	−0.10
46	15	−0.1942	−0.2323	−0.09
47	16	−0.1987	−0.2279	−0.09
48	17	−0.1951	−0.2336	−0.10
49	14	−0.2092	−0.2231	−0.10
50	18	−0.1941	−0.2358	−0.10

9. Additional Tasks

9.1 Step vs. Sigmoid Activation

The Step Activation Function gives us a simple yes or no answer it is either zero or one. It does not tell us how to get to that answer. This is a problem because we use gradients to find the way to do things. The Step Activation Function does not give us any information about how to change the weights in a -layer network because its gradient is zero everywhere except at one point where we cannot even use it.

The Sigmoid function is really different from the Step Activation Function. It gives us an answer that we can use to figure out the probability of something. The Sigmoid function is also very good, for learning because it gives us a gradient that's not zero so we can use it to update the weights in a multi-layer network. This makes it a lot easier to train the network and get results. The Sigmoid function is a choice when we want to use gradient descent to train a network.

9.2 Comparison with Scikit-learn's Perceptron

Both the from-scratch implementation and `sklearn.linear_model.Perceptron` follow the same perceptron learning rule and achieved comparable performance (accuracy ≈ 0.985 on the test set), with minor differences attributable to implementation details such as sample shuffling order and internal tie-breaking during updates.

9.3 Learning Rates 0.001, 0.01, 0.1

The three learning rates all gave the result. They all had the convergence curve and the same test accuracy of 0.9855. When we use a single-layer perceptron with the step activation function the learning rates change how big each update is. They do not change the direction of the update. So the sequence of mistakes the perceptron makes at each epoch is the same. The learning rates only

change the size of the weights and bias.The learning rates are important for the perceptron to learn the weights and bias.

9.4 Why a Single Layer Perceptron Cannot Solve XOR

XOR is not linearly separable: no single straight line can separate $(0,0) \to 0$, $(1,1) \to 0$ from $(0,1) \to 1$, $(1,0) \to 1$. Since a single-layer perceptron can only represent a linear decision boundary $\mathbf{w}^T\mathbf{x}+b=0$, it cannot represent the XOR function. Solving XOR requires at least one hidden layer (a Multilayer Perceptron) with non-linear activations to construct a non-linear decision region.

9.5 Effect of Feature Normalization on Convergence

Standardizing the four features before training makes sure all inputs have similar scales. This stops features with larger raw values, like Skewness, , from overshadowing smaller-scale features like Entropy during weight updates. In this experiment, using normalized data for training led to a more stable results than if we had used raw feature values. Unnormalized inputs usually result in slower and more erratic convergence.

10. Discussion

The Single Layer Perceptron achieved a high test accuracy of 98.55 on the Banknote Authentication dataset. It had perfect recall (1.0000) for the forged class and misclassified only four authentic notes. The exploratory data analysis showed that Variance and Skewness are the most distinct features, with the highest correlation to the class label. This was visually supported by the scatter plot and the two-feature decision boundary. Although the training error did not drop to zero within 50 epochs, this indicates that the four-dimensional feature space is not perfectly linearly separable; the classifier still performed well on new data. The study on learning rate revealed that, for this step-activation perceptron, the choice of η only influenced the scale of the learned parameters and did not affect the classification result.

11. Conclusion

I built a Single Layer Perceptron from the ground up. This included initializing weights and biases, using the step activation function, performing forward propagation, and applying the perceptron learning rule. I trained and tested the model on the UCI Banknote Authentication dataset, which resulted in a test accuracy of 98.55. The experiment confirmed the theoretical limits of the perceptron, such as its inability to solve non-linearly separable problems like XOR. It also showed the model's effectiveness on a nearly linearly separable real-world dataset. Additionally, it underscored the need for preprocessing, specifically normalization, and the shift to differentiable activation functions in deeper models.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.

5. UCI Machine Learning Repository – Banknote Authentication Dataset.

6. Scikit-learn Documentation: Perceptron.